IMPLEMENT SNOWFLAKE STREAMS FOR SALES DATA PROCESSING

A retail company receives sales data in a staging table. The company wants to load this data into a final reporting table. After the initial load, only newly inserted or updated records should be processed using Snowflake Streams.

You need to create the database, tables, stream object, insert records, consume stream data, and update the final table using stream changes.

Task 1: Create Database

Create a transient database named:

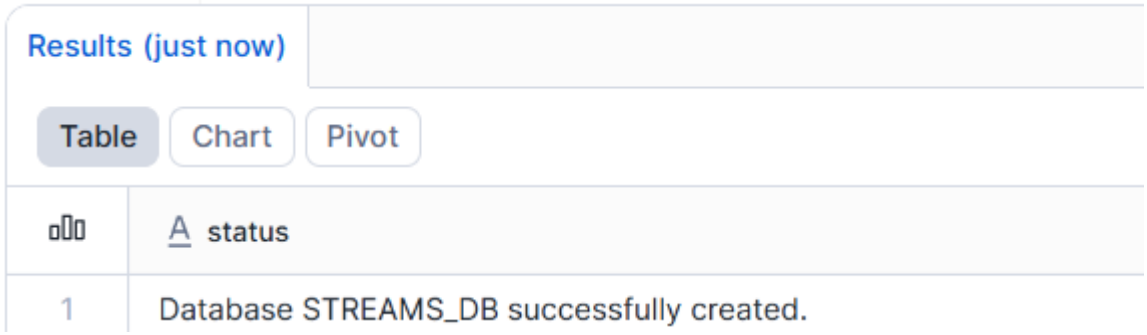
STREAMS_DB

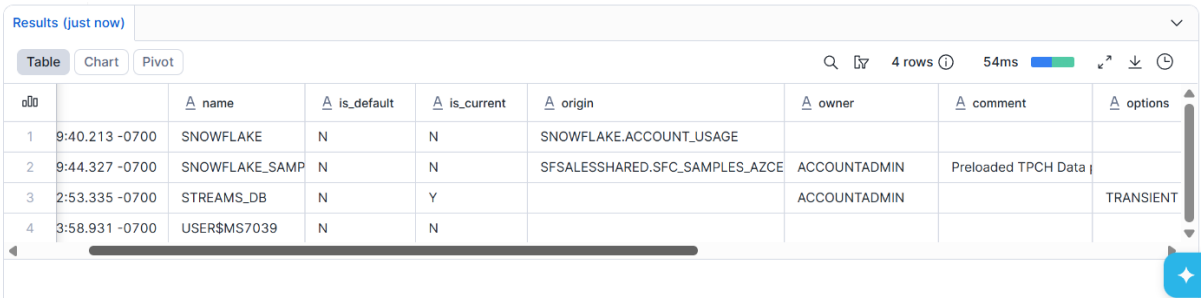
ANS:

Creating and using a transient database STREAMS_DB

```
1  CREATE OR REPLACE TRANSIENT DATABASE STREAMS_DB;
2  USE DATABASE STREAMS_DB;
```

It was successfully executed.





Task 2: Create Raw Sales Staging Table

Create a table named:

SALES_RAW_STAGING

The table should contain the following columns:

- **id**
- **product**
- **price**
- **amount**
- **store_id**

Use suitable data types.

ANS:

Creating a table SALES_RAW_STAGING

```
6  CREATE OR REPLACE TABLE SALES_RAW_STAGING (
7      id          INT,
8      product     VARCHAR(50),
9      price       DECIMAL(10, 2),
10     amount      INT,
11     store_id    INT
12 );
```

It was successfully executed.

Results (just now)

Table | Chart | Pivot

	status
1	Table SALES_RAW_STAGING successfully created.

Results (just now)

Table | Chart | Pivot — 1 row — 35ms

	created_on	name	database_name	schema_name	kind	comment	cluster_by
1	2026-05-16 08:17:12	SALES_RAW_STAGING	STREAMS_DB	PUBLIC	TRANSIENT		

Results (just now)

Table | Chart | Pivot — 5 rows — 21ms

	name	type	kind	null?	default	primary key	unique key	check	expression
1	ID	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null
2	PRODUCT	VARCHAR(50)	COLUMN	Y	null	N	N	null	null
3	PRICE	NUMBER(10,2)	COLUMN	Y	null	N	N	null	null
4	AMOUNT	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null
5	STORE_ID	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null

Task 3: Insert Initial Sales Data

Insert the following five records into SALES_RAW_STAGING:

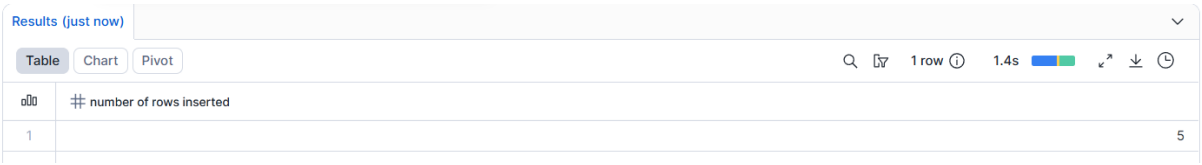
id	product	price	amount	store_id
1	Banana	1.99	1	1
2	Lemon	0.99	1	1
3	Apple	1.79	1	2
4	Orange Juice	1.89	1	
5	Cereals	5.98	2	1

ANS:

Inserting data in SALES_RAW_STAGING

```
18  INSERT INTO SALES_RAW_STAGING (id, product, price, amount, store_id)
19  VALUES
20      (1, 'Banana',       1.99, 1, 1),
21      (2, 'Lemon',        0.99, 1, 1),
22      (3, 'Apple',        1.79, 1, 2),
23      (4, 'Orange Juice', 1.89, 1, NULL),
24      (5, 'Cereals',      5.98, 2, 1);
```

It was successfully executed.



Task 4: Create Store Table

Create a table named:

STORE_TABLE

The table should contain:

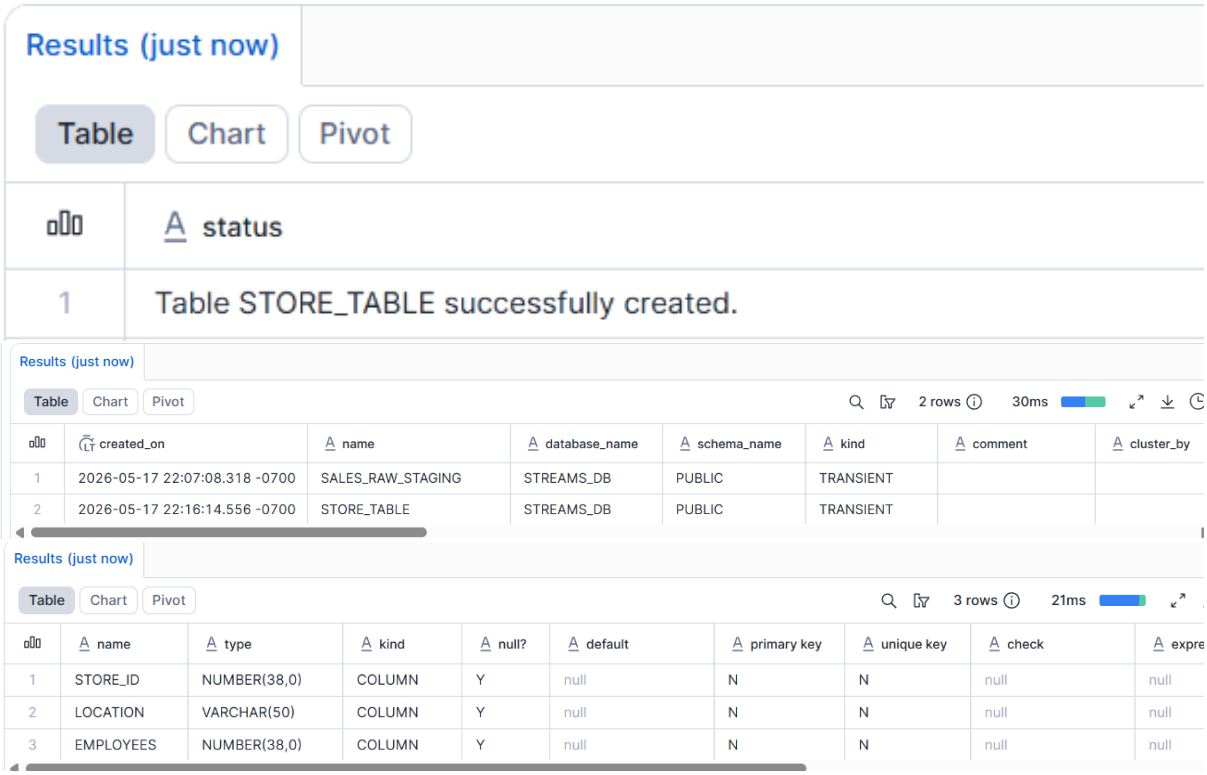
- **store_id**
- **location**
- **employees**

ANS:

Creating a table STORE_TABLE

```
28      CREATE OR REPLACE TABLE STORE_TABLE (
29          store_id  INT,
30          location  VARCHAR(50),
31          employees INT
32      );
```

It was successfully executed.



Results (just now)

Table | Chart | Pivot

	status
1	Table STORE_TABLE successfully created.

Results (just now)

Table | Chart | Pivot — 2 rows — 30ms

	created_on	name	database_name	schema_name	kind	comment	cluster_by
1	2026-05-17 22:07:08.318 -0700	SALES_RAW_STAGING	STREAMS_DB	PUBLIC	TRANSIENT		
2	2026-05-17 22:16:14.556 -0700	STORE_TABLE	STREAMS_DB	PUBLIC	TRANSIENT		

Results (just now)

Table | Chart | Pivot — 3 rows — 21ms

	name	type	kind	null?	default	primary key	unique key	check	expre
1	STORE_ID	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null
2	LOCATION	VARCHAR(50)	COLUMN	Y	null	N	N	null	null
3	EMPLOYEES	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null

Task 5: Insert Store Data

Insert the following records into STORE_TABLE:

store_id location employees

1 Chicago 33

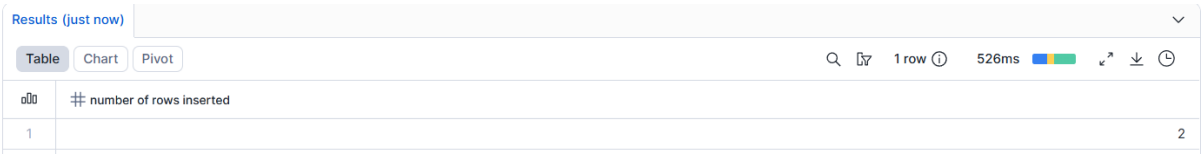
2 London 12

ANS:

Inserting data in STORE_TABLE

```
38      INSERT INTO STORE_TABLE (store_id, location, employees)
39      VALUES
40          (1, 'Chicago', 33),
41          (2, 'London',  12);
```

It was successfully executed.



Task 6: Create Final Sales Table

Create a table named:

SALES_FINAL_TABLE

The table should contain:

- **id**
- **product**
- **price**
- **amount**
- **store_id**
- **location**
- **employees**

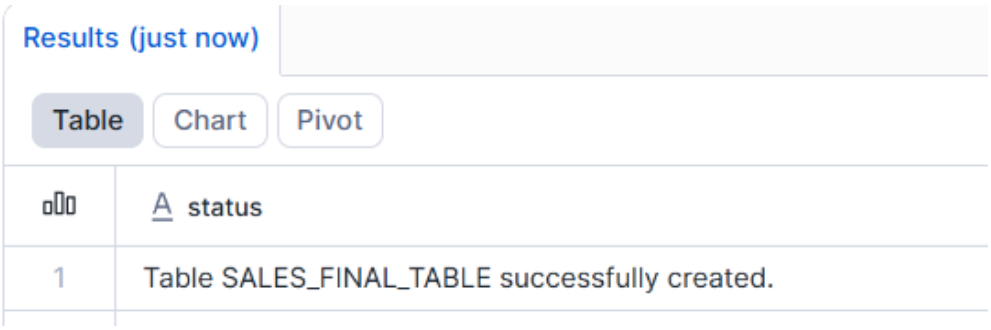
This table will store sales data along with store location and employee details.

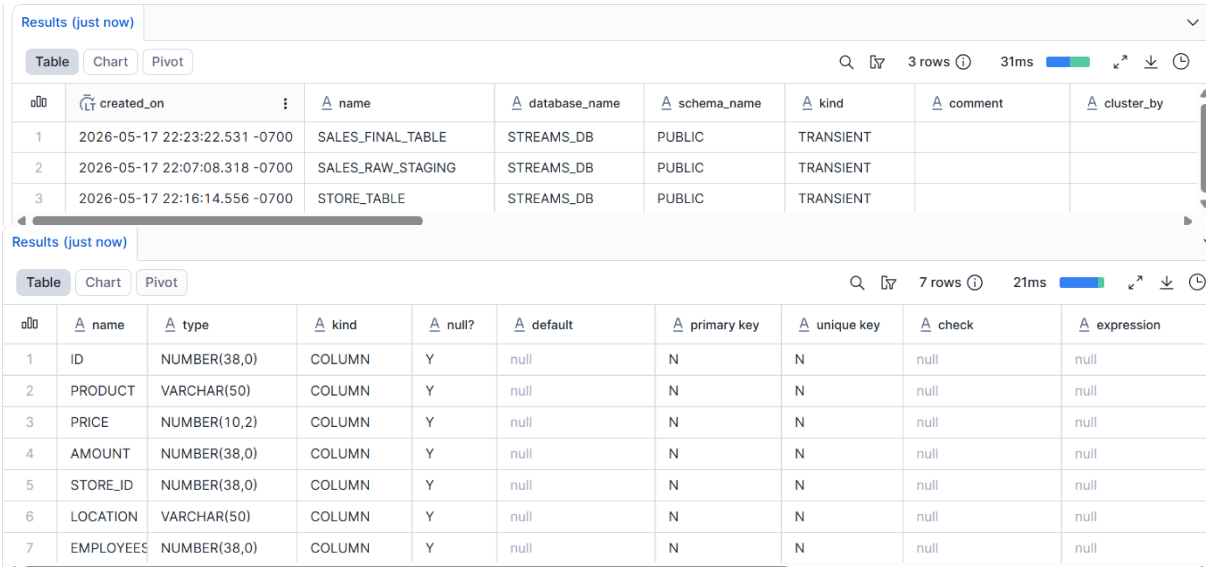
ANS:

Creating a table SALES_FINAL_TABLE

```
45  CREATE OR REPLACE TABLE SALES_FINAL_TABLE (
46      id        INT,
47      product   VARCHAR(50),
48      price     DECIMAL(10, 2),
49      amount    INT,
50      store_id  INT,
51      location  VARCHAR(50),
52      employees INT
53  );
```

It was successfully executed.





Results (just now)

Table | Chart | Pivot — 3 rows — 31ms

	created_on	name	database_name	schema_name	kind	comment	cluster_by
1	2026-05-17 22:23:22.531 -0700	SALES_FINAL_TABLE	STREAMS_DB	PUBLIC	TRANSIENT		
2	2026-05-17 22:07:08.318 -0700	SALES_RAW_STAGING	STREAMS_DB	PUBLIC	TRANSIENT		
3	2026-05-17 22:16:14.556 -0700	STORE_TABLE	STREAMS_DB	PUBLIC	TRANSIENT		

Results (just now)

Table | Chart | Pivot — 7 rows — 21ms

	name	type	kind	null?	default	primary key	unique key	check	expression
1	ID	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null
2	PRODUCT	VARCHAR(50)	COLUMN	Y	null	N	N	null	null
3	PRICE	NUMBER(10,2)	COLUMN	Y	null	N	N	null	null
4	AMOUNT	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null
5	STORE_ID	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null
6	LOCATION	VARCHAR(50)	COLUMN	Y	null	N	N	null	null
7	EMPLOYEES	NUMBER(38,0)	COLUMN	Y	null	N	N	null	null

Task 7: Perform Initial Full Load

Load data from SALES_RAW_STAGING into SALES_FINAL_TABLE.

While loading, join:

SALES_RAW_STAGING

with:

STORE_TABLE

using STORE_ID.

ANS:

Inserting data from SALES_RAW_STAGING in SALES_FINAL_TABLE

```
59  INSERT INTO SALES_FINAL_TABLE
60  SELECT
61      sr.id,
62      sr.product,
63      sr.price,
64      sr.amount,
65      sr.store_id,
66      st.location,
67      st.employees
68  FROM SALES_RAW_STAGING sr
69  LEFT JOIN STORE_TABLE st
70      ON sr.store_id = st.store_id;
71
```

It was successfully executed.

Results (just now)

Table | Chart | Pivot — 1 row — 210ms

	number of rows inserted
1	5

Task 8: Verify Initial Load

Display data from:

SALES_RAW_STAGING;

STORE_TABLE;

SALES_FINAL_TABLE;

Confirm that SALES_FINAL_TABLE contains 5 records.

Stream Implementation

ANS:

Verifying all the records SALES_RAW_STAGING, STORE_TABLE and SALES_FINAL_TABLE

```
72    SELECT * FROM SALES_RAW_STAGING;
73    SELECT * FROM STORE_TABLE;
74    SELECT * FROM SALES_FINAL_TABLE;
```

It was successfully executed.

SALES_RAW_STAGING

Results (just now)

Table | Chart | Pivot — 5 rows — 23ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	Banana	1.99	1	1
2	2	Lemon	0.99	1	1
3	3	Apple	1.79	1	2
4	4	Orange Juice	1.89	1	null
5	5	Cereals	5.98	2	1

STORE_TABLE

Results (just now)

Table | Chart | Pivot — 2 rows — 73ms

	STORE_ID	LOCATION	EMPLOYEES
1	1	Chicago	33
2	2	London	12

SALES_FINAL_TABLE

Results (just now)

Table | Chart | Pivot — 5 rows — 69ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Banana	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33

Task 9: Create Stream Object

Create a stream named:

SALES_STREAM

on top of:

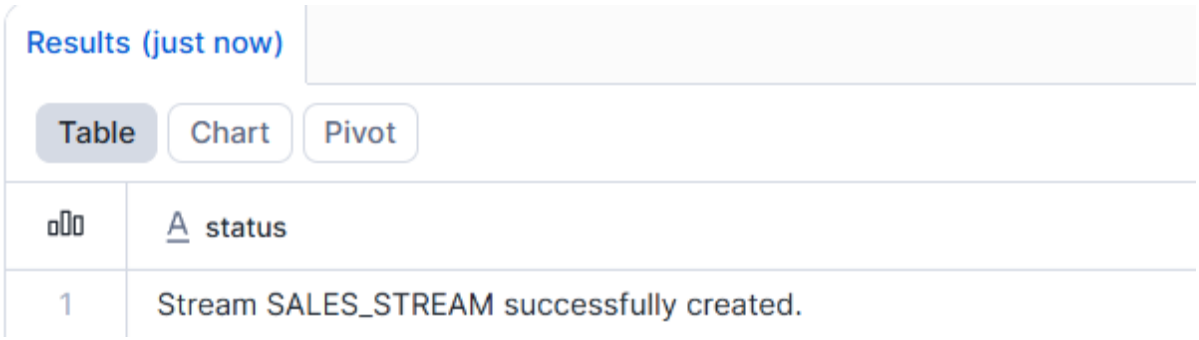
SALES_RAW_STAGING

ANS:

Creating a stream SALES_STREAM

```
78    CREATE OR REPLACE STREAM SALES_STREAM
79        ON TABLE SALES_RAW_STAGING;
```

It was successfully executed.



Task 10: View Stream Information

Execute commands to:

Show all streams.

Describe the SALES_STREAM.

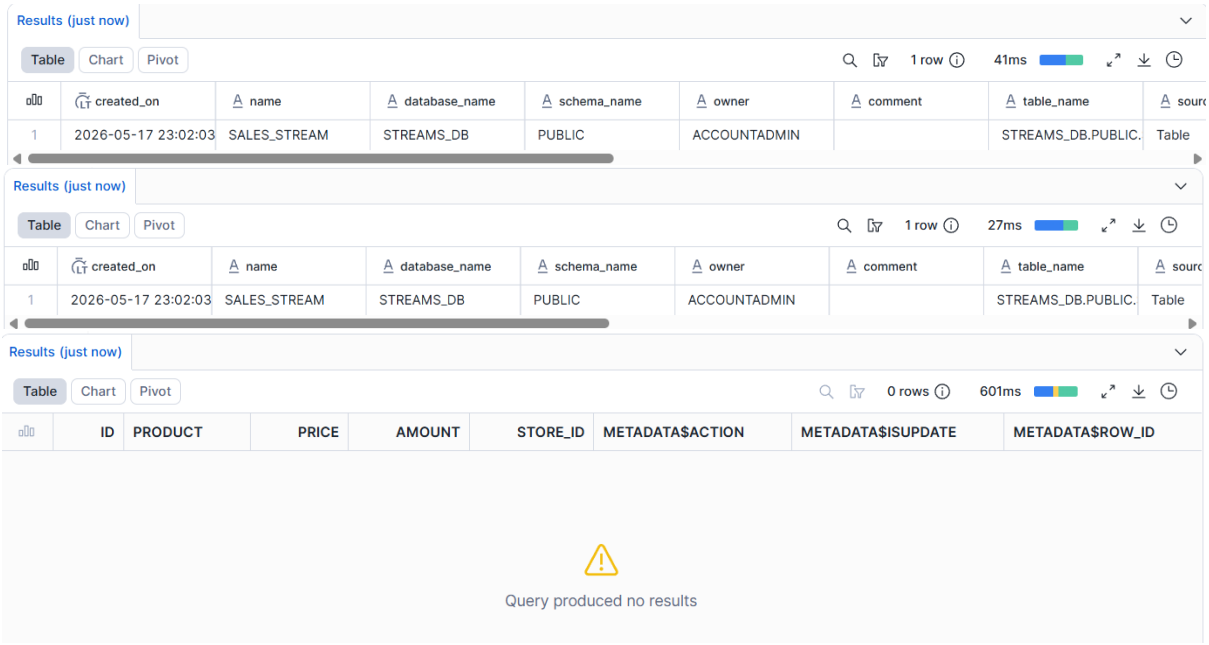
Select data from SALES_STREAM.

ANS:

Along with showing all streams, describe and select data from SALES_STREAM

```
81    SHOW STREAMS;
82    DESC STREAM SALES_STREAM;
83    SELECT * FROM SALES_STREAM;
```

It was successfully executed.



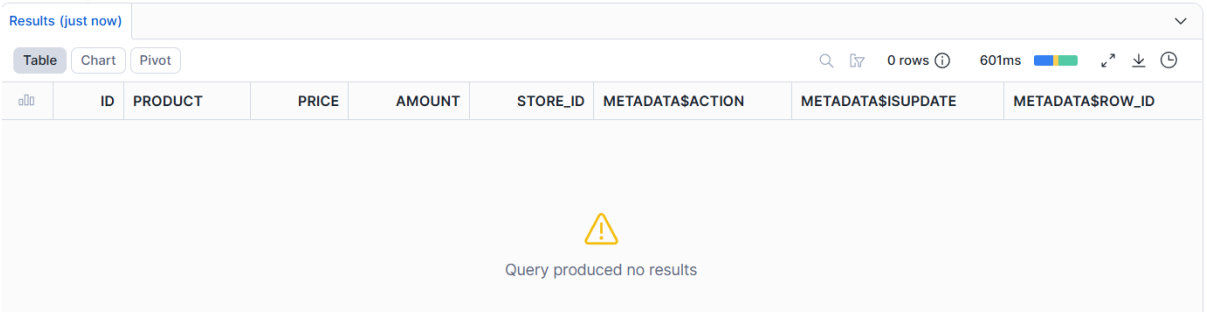
Answer this question:

Why is SALES_STREAM empty immediately after creation?

ANS:

A Snowflake stream records only changes (DML) that occur AFTER the stream's creation offset (timestamp). Since no INSERT, UPDATE, or DELETE has been performed on SALES_RAW_STAGING after the stream was created, there are no change records to capture yet.

The existing rows already in the table are NOT part of the stream — the stream starts watching from this point forward.



Task 11: Insert New Sales Records

Insert the following two records into SALES_RAW_STAGING:

id	product	price	amount	store_id
6	Mango	1.99	1	2
7	Garlic 0.99	1	1	

ANS:

Inserting data in SALES_RAW_STAGING

```
81  INSERT INTO SALES_RAW_STAGING (id, product, price, amount, store_id)
82  VALUES
83      (6, 'Mango',  1.99, 1, 2),
84      (7, 'Garlic', 0.99, 1, 1);
```

It was successfully executed.

Task 12: Check Stream Data

Query SALES_STREAM.

Observe the new records and metadata columns:

METADATA$ACTION

METADATA$ISUPDATE

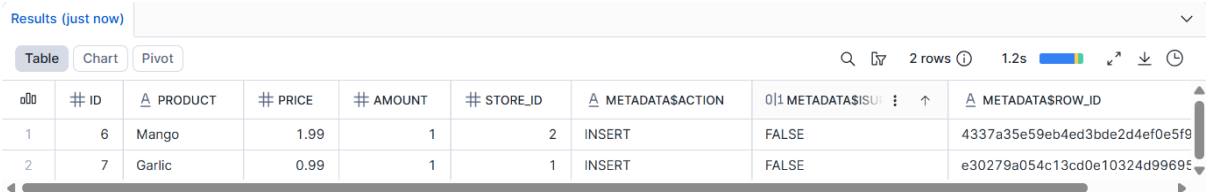
METADATA$ROW_ID

ANS:

Checking stream data in SALES_STREAM

```
86  SELECT * FROM SALES_STREAM;
```

It was successfully executed.



Answer this question:

What value do you see in METADATA$ACTION for newly inserted records?

ANS:

METADATA$ACTION = 'INSERT'

For newly inserted rows the stream shows:

- METADATA$ACTION = 'INSERT'
- METADATA$ISUPDATE = FALSE
- METADATA$ROW_ID = a unique internal row identifier (SHA hash)

Task 13: Verify Source and Target Tables

Check data in:

SALES_RAW_STAGING;

SALES_FINAL_TABLE;

ANS:

Verifying all the records SALES_RAW_STAGING and SALES_FINAL_TABLE

```
88    SELECT * FROM SALES_RAW_STAGING;
89    SELECT * FROM SALES_FINAL_TABLE;
```

It was successfully executed.

SALES_RAW_STAGING:

Results (just now)

Table | Chart | Pivot — 7 rows — 1.3s

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	Banana	1.99	1	1
2	2	Lemon	0.99	1	1
3	3	Apple	1.79	1	2
4	4	Orange Juice	1.89	1	null
5	5	Cereals	5.98	2	1
6	6	Mango	1.99	1	2
7	7	Garlic	0.99	1	1

SALES_FINAL_TABLE:

Results (just now)

Table | Chart | Pivot — 5 rows — 87ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Banana	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33

Answer this question:

Are the new records available in SALES_FINAL_TABLE before consuming the stream?

ANS:

NO. The new records (Mango, Garlic) are visible in SALES_RAW_STAGING but are NOT yet in SALES_FINAL_TABLE. Stream consumption is an explicit, manual step — data does not automatically propagate.

Task 14: Consume Stream Data

Insert new records from SALES_STREAM into SALES_FINAL_TABLE.

While inserting, join the stream with STORE_TABLE using STORE_ID.

ANS:

Inserting data from SALES_STREAM to SALES_FINAL_TABLE

```
91  INSERT INTO SALES_FINAL_TABLE
92  SELECT
93      ss.id,
94      ss.product,
95      ss.price,
96      ss.amount,
97      ss.store_id,
98      st.location,
99      st.employees
100 FROM SALES_STREAM ss
101 LEFT JOIN STORE_TABLE st
102     ON ss.store_id = st.store_id
103 WHERE ss.METADATA$ACTION = 'INSERT'
104   AND ss.METADATA$ISUPDATE = FALSE;
```

It was successfully executed.

Task 15: Verify Stream After Consumption

Query:

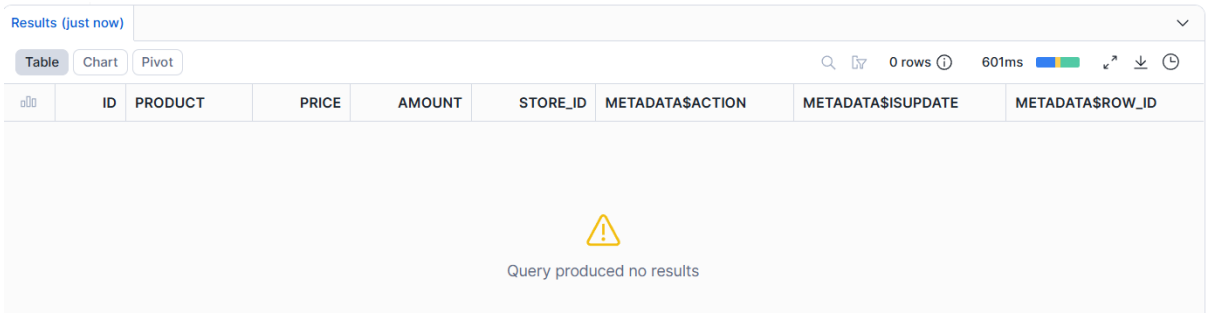
SELECT * FROM SALES_STREAM;

ANS:

Checking stream data in SALES_STREAM

```
106 SELECT * FROM SALES_STREAM;
```

It was successfully executed.

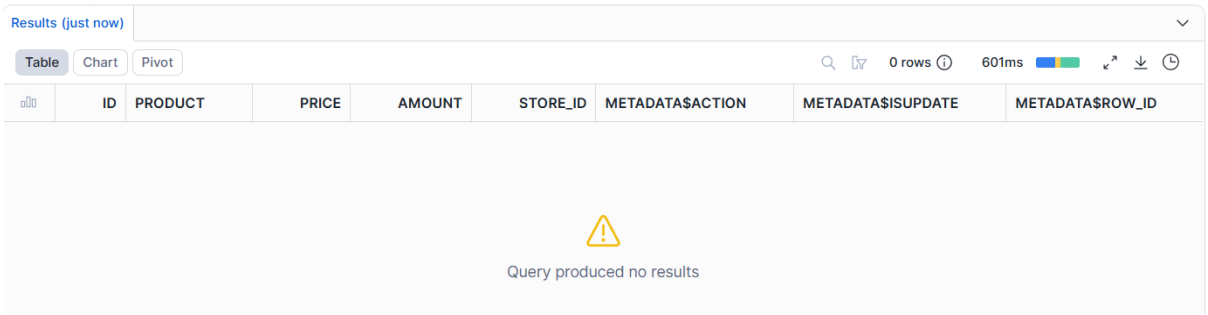


Answer this question:

Why did the stream become empty after inserting data into SALES_FINAL_TABLE?

ANS:

Snowflake streams use an OFFSET pointer to track which changes have been consumed. When a DML statement successfully reads from the stream (INSERT … SELECT FROM SALES_STREAM), Snowflake advances the offset past all records that were processed. Those records are no longer returned by the stream — they have been "consumed". The stream will only show new changes that occur after the current offset.



Task 16: Insert More Sales Records

Insert the following two records into SALES_RAW_STAGING:

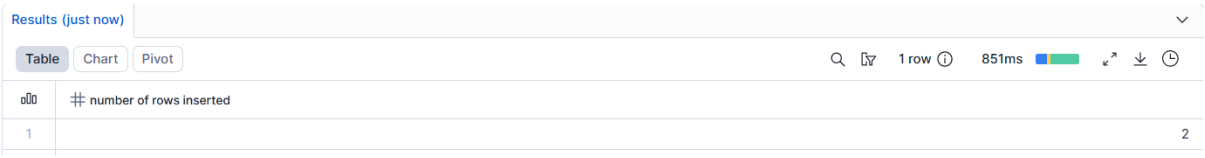
id	product	price	amount	store_id
8	Paprika	4.99	1	2
9	Tomato	3.99	1	2

ANS:

Inserting data in SALES_RAW_STAGING

```
108  INSERT INTO SALES_RAW_STAGING (id, product, price, amount, store_id)
109  VALUES
110      (8, 'Paprika', 4.99, 1, 2),
111      (9, 'Tomato',  3.99, 1, 2);
```

It was successfully executed.



Task 17: Consume Second Stream Data

Consume the newly captured stream records and insert them into SALES_FINAL_TABLE.

ANS:

Inserting data in SALES_FINAL_TABLE from the newly captured stream records

```
INSERT INTO SALES_FINAL_TABLE
SELECT
    ss.id,
    ss.product,
    ss.price,
    ss.amount,
    ss.store_id,
    st.location,
    st.employees
FROM SALES_STREAM ss
LEFT JOIN STORE_TABLE st
    ON ss.store_id = st.store_id
WHERE ss.METADATA$ACTION = 'INSERT'
  AND ss.METADATA$ISUPDATE = FALSE;
```

It was successfully executed.

Task 18: Verify Final Insert Output

Display data from:

SALES_FINAL_TABLE;

SALES_RAW_STAGING;

SALES_STREAM;

ANS:

Verifying all the records SALES_FINAL_TABLE, SALES_RAW_STAGING and SALES_STREAM

```
128    SELECT * FROM SALES_FINAL_TABLE;
129    SELECT * FROM SALES_RAW_STAGING;
130    SELECT * FROM SALES_STREAM;
```

It was successfully executed.

SALES_FINAL_TABLE

Results (just now)

Table | Chart | Pivot — 9 rows — 80ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Banana	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33
6	7	Garlic	0.99	1	1	Chicago	33
7	6	Mango	1.99	1	2	London	12
8	8	Paprika	4.99	1	2	London	12
9	9	Tomato	3.99	1	2	London	12

SALES_RAW_STAGING

Results (just now)

Table | Chart | Pivot — 9 rows — 763ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	Banana	1.99	1	1
2	2	Lemon	0.99	1	1
3	3	Apple	1.79	1	2
4	4	Orange Juice	1.89	1	null
5	5	Cereals	5.98	2	1
6	6	Mango	1.99	1	2
7	7	Garlic	0.99	1	1
8	8	Paprika	4.99	1	2
9	9	Tomato	3.99	1	2

SALES_STREAM

Results (just now)

Table | Chart | Pivot — 0 rows — 124ms

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA$ACTION	METADATA$ISUPDATE	METADATA$ROW_ID

Query produced no results

Answer this question:

How many records are now available in SALES_FINAL_TABLE?

ANS:

9 records total:

- IDs 1–5 → initial full load
- IDs 6–7 → first stream consumption (Mango, Garlic)
- IDs 8–9 → second stream consumption (Paprika, Tomato)

Results (just now)

Table | Chart | Pivot — 9 rows — 80ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Banana	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33
6	7	Garlic	0.99	1	1	Chicago	33
7	6	Mango	1.99	1	2	London	12
8	8	Paprika	4.99	1	2	London	12
9	9	Tomato	3.99	1	2	London	12

Task 19: Check Existing Data Before Update

Display records from:

SALES_RAW_STAGING;

SALES_STREAM;

ANS:

Verifying all the records SALES_RAW_STAGING and SALES_STREAM

```
132    SELECT * FROM SALES_RAW_STAGING;
133    SELECT * FROM SALES_STREAM;
```

It was successfully executed.

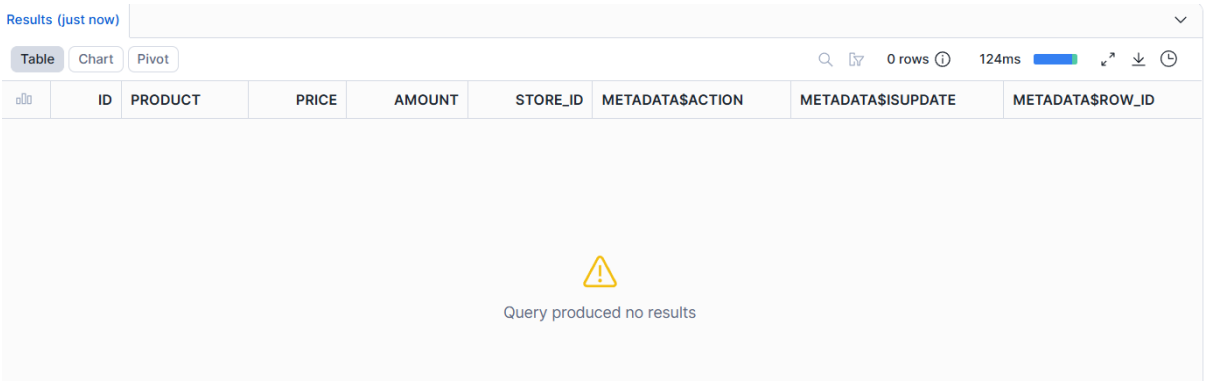
SALES_RAW_STAGING

Results (just now)

Table | Chart | Pivot — 9 rows — 32ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	Banana	1.99	1	1
2	2	Lemon	0.99	1	1
3	3	Apple	1.79	1	2
4	4	Orange Juice	1.89	1	null
5	5	Cereals	5.98	2	1
6	6	Mango	1.99	1	2
7	7	Garlic	0.99	1	1
8	8	Paprika	4.99	1	2
9	9	Tomato	3.99	1	2

SALES_STREAM



Task 20: Perform First Update

Update the product name:

Banana

to:

Potato

in SALES_RAW_STAGING.

ANS:

Updating data in SALES_RAW_STAGING

```
135  UPDATE SALES_RAW_STAGING
136  SET product = 'Potato'
137  WHERE product = 'Banana';
```

It was successfully executed.

Task 21: Check Stream After Update

Query:

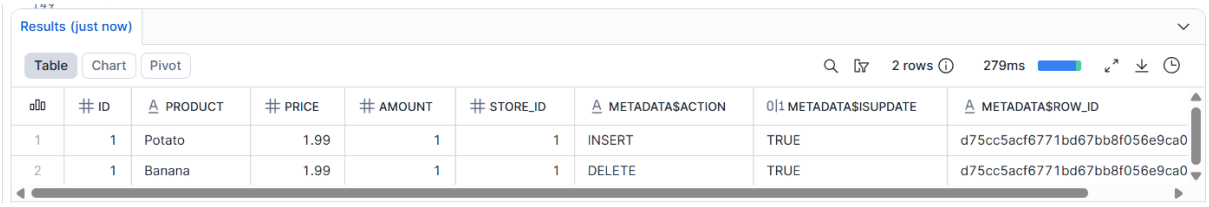
SELECT * FROM SALES_STREAM;

ANS:

Checking stream data in SALES_STREAM

```
139  SELECT * FROM SALES_STREAM;
```

It was successfully executed.



	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA$ACTION	METADATA$ISUPDATE	METADATA$ROW_ID
1	1	Potato	1.99	1	1	INSERT	TRUE	d75cc5acf6771bd67bb8f056e9ca0
2	1	Banana	1.99	1	1	DELETE	TRUE	d75cc5acf6771bd67bb8f056e9ca0

Answer this question:

How does Snowflake represent an UPDATE operation inside a stream?

ANS:

Snowflake does NOT store a single "UPDATE" record. Instead it stores TWO rows for the same METADATA$ROW_ID:

- Row 1: METADATA$ACTION = 'DELETE', METADATA$ISUPDATE = TRUE: represents the OLD version of the row (before the change)
- Row 2: METADATA$ACTION = 'INSERT', METADATA$ISUPDATE = TRUE: represents the NEW version of the row (after the change)

To apply updates downstream, we match on ACTION = 'INSERT' AND ISUPDATE = TRUE (the new value row).

Task 22: Consume Update Using MERGE

Use a MERGE statement to update the corresponding record in SALES_FINAL_TABLE.

Condition:

Match records using ID

Update only records where METADATA$ACTION = 'INSERT'

and METADATA$ISUPDATE = TRUE

Update the following columns:

- **product**
- **price**
- **amount**
- **store_id**

ANS:

With help of MERGE statement, updating data in SALES_FINAL_TABLE

```sql
MERGE INTO SALES_FINAL_TABLE AS tgt
USING (
    SELECT *
    FROM SALES_STREAM
    WHERE METADATA$ACTION    = 'INSERT'
      AND METADATA$ISUPDATE = TRUE
) AS src
ON tgt.id = src.id
WHEN MATCHED THEN
    UPDATE SET
        tgt.product  = src.product,
        tgt.price    = src.price,
        tgt.amount   = src.amount,
        tgt.store_id = src.store_id;
```

It was successfully executed.

Task 23: Verify First Update

Display data from:

SALES_FINAL_TABLE;

SALES_RAW_STAGING;

SALES_STREAM;

ANS:

Verifying all the records SALES_FINAL_TABLE, SALES_RAW_STAGING and SALES_STREAM

```sql
SELECT * FROM SALES_FINAL_TABLE;
SELECT * FROM SALES_RAW_STAGING;
SELECT * FROM SALES_STREAM;
```

It was successfully executed.

SALES_FINAL_TABLE

Results (just now)

Table | Chart | Pivot — 9 rows — 1.9s

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Potato	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33
6	7	Garlic	0.99	1	1	Chicago	33
7	6	Mango	1.99	1	2	London	12
8	8	Paprika	4.99	1	2	London	12
9	9	Tomato	3.99	1	2	London	12

SALES_RAW_STAGING

Results (just now)

Table | Chart | Pivot — 9 rows — 923ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	Potato	1.99	1	1
2	2	Lemon	0.99	1	1
3	3	Apple	1.79	1	2
4	4	Orange Juice	1.89	1	null
5	5	Cereals	5.98	2	1
6	6	Mango	1.99	1	2
7	7	Garlic	0.99	1	1
8	8	Paprika	4.99	1	2
9	9	Tomato	3.99	1	2

SALES_STREAM

Results (just now)

Table | Chart | Pivot — 0 rows — 124ms

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA$ACTION	METADATA$ISUPDATE	METADATA$ROW_ID

Query produced no results

Answer this question:

Is Banana changed to Potato in SALES_FINAL_TABLE?

ANS:

YES. The MERGE matched on id = 1 and updated the product column from 'Banana' to 'Potato' in SALES_FINAL_TABLE.

Results (just now)

Table | Chart | Pivot — 9 rows — 1.9s

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Potato	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33
6	7	Garlic	0.99	1	1	Chicago	33
7	6	Mango	1.99	1	2	London	12
8	8	Paprika	4.99	1	2	London	12
9	9	Tomato	3.99	1	2	London	12

Task 24: Perform Second Update

Update the product name:

Apple

to:

Green apple

in SALES_RAW_STAGING.

ANS:

Updating data in SALES_RAW_STAGING

```
160  UPDATE SALES_RAW_STAGING
161  SET product = 'Green apple'
162  WHERE product = 'Apple';
```

It was successfully executed.

Results (1 minute ago)

Table | Chart | Pivot — 1 row — 500ms

	number of rows updated	number of multi-joined rows updated
1	1	0

Task 25: Consume Second Update Using MERGE

Use a MERGE statement to update the changed record in SALES_FINAL_TABLE.

ANS:

With help of MERGE statement, updating data in SALES_FINAL_TABLE

```
164  MERGE INTO SALES_FINAL_TABLE AS tgt
165  USING (
166      SELECT *
167      FROM SALES_STREAM
168      WHERE METADATA$ACTION    = 'INSERT'
169        AND METADATA$ISUPDATE = TRUE
170  ) AS src
171  ON tgt.id = src.id
172  WHEN MATCHED THEN
173      UPDATE SET
174          tgt.product  = src.product,
175          tgt.price    = src.price,
176          tgt.amount   = src.amount,
177          tgt.store_id = src.store_id;
```

It was successfully executed.

Task 26: Verify Final Output

Display data from:

SALES_FINAL_TABLE;

SALES_RAW_STAGING;

SALES_STREAM;

ANS:

Verifying all the records SALES_FINAL_TABLE, SALES_RAW_STAGING and SALES_STREAM

```
179  SELECT * FROM SALES_FINAL_TABLE;
180  SELECT * FROM SALES_RAW_STAGING;
181  SELECT * FROM SALES_STREAM;
```

It was successfully executed.

SALES_FINAL_TABLE

Results (just now)

Table | Chart | Pivot — 9 rows — 114ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Potato	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Green apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33
6	7	Garlic	0.99	1	1	Chicago	33
7	6	Mango	1.99	1	2	London	12
8	8	Paprika	4.99	1	2	London	12
9	9	Tomato	3.99	1	2	London	12

SALES_RAW_STAGING

Results (just now)

Table | Chart | Pivot — 9 rows — 76ms

	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	Potato	1.99	1	1
2	2	Lemon	0.99	1	1
3	3	Green apple	1.79	1	2
4	4	Orange Juice	1.89	1	null
5	5	Cereals	5.98	2	1
6	6	Mango	1.99	1	2
7	7	Garlic	0.99	1	1
8	8	Paprika	4.99	1	2
9	9	Tomato	3.99	1	2

SALES_STREAM

Results (just now)

Table | Chart | Pivot — 0 rows — 124ms

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA$ACTION	METADATA$ISUPDATE	METADATA$ROW_ID

Query produced no results

Answer this question:

Is Apple changed to Green apple in SALES_FINAL_TABLE?

ANS:

YES. The MERGE matched on id = 3 (the Apple row) and updated the product column to 'Green apple' in SALES_FINAL_TABLE.

Final state of SALES_FINAL_TABLE (9 records):

- id=1 Potato (was Banana — updated Task 22)
- id=2 Lemon
- id=3 Green apple (was Apple — updated Task 25)

- id=4 Orange Juice (store_id NULL → location/employees NULL)
- id=5 Cereals
- id=6 Mango
- id=7 Garlic
- id=8 Paprika
- id=9 Tomato

SALES_STREAM is empty — all changes have been consumed.